

Sprint 7: Form Fundamentals — Full Structured Notes

Sprint 7 Goal

The goal of Sprint 7 is to learn how to create basic HTML forms.

A form is used when a web page needs to collect information from a user.

Examples of information a form can collect:

- Full name
- Email address
- Phone number
- Search text
- Login details
- Registration details
- Contact message

In this sprint, the main focus is not styling, validation, or buttons. The main focus is learning how to build the basic structure of a form correctly.

By the end of Sprint 7, you should understand:

- What a `form` element is
- What the `action` attribute does
- What an `input` element is
- Why `input` is a void element
- How to use basic input types like `text` and `email`
- What a `label` does
- How to connect labels to inputs
- The difference between implicit and explicit label association
- The difference between `id` and `name`
- What a `placeholder` does
- Why placeholder text must not replace a label
- Why labels are important for accessibility
- How to debug common beginner form mistakes

1. Where Sprint 7 Fits in Your HTML Progress

Before Sprint 7, you learned how to build the structure of a web page.

Sprint 1

You learned basic HTML syntax:

```
<h1>My Page</h1>
<p>This is a paragraph.</p>

```

You learned:

- Elements
- Tags
- Attributes
- Void elements
- `id`
- `class`
- Basic image structure

Sprint 2

You learned how to write a full HTML document:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My Page</title>
</head>
<body>
  <h1>My Page</h1>
</body>
</html>
```

You learned:

- `DOCTYPE`
- `html`
- `head`
- `body`
- Metadata
- Page title
- Visible page content

Sprint 3

You learned how to mark text based on meaning:

```
<p><strong>Warning:</strong> Do not share your password.</p>
<p>The term <i>semantic HTML</i> means HTML with meaning.</p>
```

Sprint 4

You learned links and navigation:

```
<nav>
  <a href="index.html">Home</a>
  <a href="about.html">About</a>
  <a href="contact.html">Contact</a>
</nav>
```

Sprint 5

You learned media:

```

```

Sprint 6

You learned semantic page structure:

```
<header>
  <h1>Code Learning Hub</h1>
</header>

<main>
  <section>
    <h2>Featured Courses</h2>
    <p>Learn HTML step by step.</p>
  </section>
</main>
```

Sprint 7

Now you are learning forms.

Simple idea:

Sprints 1 to 6 taught you how to display and organize content.
Sprint 7 teaches you how to collect user input.

2. What Is a Form?

A form is a part of a web page that collects information from the user.

Example situations:

- A user types their name
- A user enters an email address
- A user searches for something
- A user fills a registration form
- A user sends a contact message

The basic HTML element for a form is:

```
<form>  
</form>
```

A form usually contains form controls, such as inputs.

Example:

```
<form action="">  
  <label for="full-name">Full Name:</label>  
  <input type="text" id="full-name" name="full-name">  
</form>
```

Simple meaning:

```
<form> = the area that collects user data  
<label> = the name or explanation for an input  
<input> = the field where the user enters data  
action = where the form data will be sent
```

3. The `form` Element

The `form` element groups form controls together.

Basic pattern:

```
<form action="">
  <!-- form fields go here -->
</form>
```

The opening tag is:

```
<form action="">
```

The closing tag is:

```
</form>
```

The content inside the form can include:

- Labels
- Inputs
- Buttons
- Help text
- Field instructions

In Sprint 7, we mainly focus on labels and inputs.

4. The `action` Attribute

The `action` attribute tells the browser where to send the form data when the form is submitted.

Example:

```
<form action="/register">
</form>
```

Simple meaning:

When the user submits the form, send the form data to /register.

Another example:

```
<form action="/contact">  
</form>
```

Simple meaning:

When the user submits this form, send the data to /contact.

For beginner practice, you can use an empty action:

```
<form action="">  
</form>
```

Simple meaning:

This form is not connected to a real server destination yet.

Important beginner rule:

Always include action in your practice forms, even if it is empty for now.

5. What Is an `input` Element?

The `input` element creates a field where the user can enter information.

Example:

```
<input type="text">
```

This creates a normal text field.

Another example:

```
<input type="email">
```

This creates a field intended for an email address.

Simple meaning:

```
input = a form control that collects user data
```

6. `input` Is a Void Element

The `input` element is a void element.

A void element does not wrap content.

That means `input` does not have a closing tag.

Correct:

```
<input type="text">
```

Wrong:

```
<input type="text"></input>
```

Why is the second one wrong?

Because `input` cannot contain text or child elements.

Simple rule:

```
input is void.  
Do not write </input>.
```

Other void elements you already learned include:

```
  
<br>
```

7. The `type` Attribute

The `type` attribute tells the browser what kind of data the input expects.

Basic pattern:

```
<input type="value">
```

Example:

```
<input type="text">
```

This means:

```
This input expects normal text.
```

Example:

```
<input type="email">
```

This means:

```
This input expects an email address.
```

In Sprint 7, focus mainly on:

- `type="text"`
- `type="email"`

Validation comes in Sprint 8, so do not worry too much about validation yet.

8. Text Input

A text input is used for normal text.

Example:


```
<input type="text">
```

Use text input for things like:

- Full name
- Username
- City
- Country
- Search text

Example with a label:

```
<label for="full-name">Full Name:</label>  
<input type="text" id="full-name" name="full-name">
```

Simple meaning:

The user can type their full name into this field.

9. Email Input

An email input is used for email addresses.

Example:

```
<input type="email">
```

Better version with label, id, and name:

```
<label for="email">Email Address:</label>  
<input type="email" id="email" name="email">
```

Simple meaning:

The user should enter an email address in this field.

Important note:

Some browsers may check whether the typed text looks like an email address, but Sprint 7 does not focus deeply on validation. That comes in Sprint 8.

10. What Is a Label?

A `label` gives a name or description to an input.

Bad example:

```
<input type="text">
```

Problem:

```
The user does not know what this field is for.
```

Good example:

```
<label for="full-name">Full Name:</label>
<input type="text" id="full-name" name="full-name">
```

Now the user understands:

```
This field is for the full name.
```

Simple rule:

```
Every input should have a clear label.
```

11. Why Labels Are Important

Labels are important because they help many types of users.

1. Labels help normal users

A user can quickly understand what to type.

Example:

```
<label for="email">Email Address:</label>  
<input type="email" id="email" name="email">
```

The user knows this field is for an email address.

2. Labels help screen reader users

Screen readers can announce the label when the user focuses on the input.

Without a label, a screen reader user may hear only something like:

Edit text

That is not clear.

With a label, the screen reader can communicate something like:

Full Name, edit text

That is much clearer.

3. Labels make forms easier to click

When a label is correctly connected to an input, clicking the label can focus the input.

This is helpful for usability.

Simple rule:

Labels are not optional decoration.
Labels explain the purpose of the field.

12. Two Ways to Connect a Label to an Input

There are two main ways to connect a label and an input:

1. Explicit label association
2. Implicit label association

Both methods are valid.

13. Explicit Label Association

Explicit label association uses:

- `for` on the label
- `id` on the input

The value of `for` must match the value of `id`.

Example:

```
<label for="email">Email Address:</label>
<input type="email" id="email" name="email">
```

Here:

```
for="email"
id="email"
```

They match.

That means the label is connected to the input.

Simple meaning:

```
This label belongs to the input with id="email".
```

Another example:

```
<label for="full-name">Full Name:</label>
<input type="text" id="full-name" name="full-name">
```

Here:

```
for="full-name"
id="full-name"
```

They match.

Simple rule:

For explicit labels, label for must match input id.

14. Why Explicit Labels Are Good for Beginners

Explicit labels are easy to read.

Example:

```
<label for="username">Username:</label>
<input type="text" id="username" name="username">
```

A beginner can clearly see the connection:

```
label for="username"
connects to
input id="username"
```

This is why explicit labels are a strong beginner pattern.

Recommended beginner style:

```
<label for="field-id">Field Name:</label>
<input type="text" id="field-id" name="field-id">
```

15. Implicit Label Association

Implicit label association means the input is placed inside the label.

Example:

```
<label>
  Full Name:
  <input type="text" name="full-name">
</label>
```

Here, the label wraps the input.

The browser understands that the text inside the label belongs to the input inside it.

Simple meaning:

The input is inside the label, so they are connected.

Another example:

```
<label>
  Email Address:
  <input type="email" name="email" placeholder="example@email.com">
</label>
```

This is valid.

However, for clean beginner work, explicit labels are usually easier to understand and debug.

16. Explicit vs Implicit Labels

Type	Pattern	Simple Meaning
Explicit label	<code>label for</code> matches <code>input id</code>	The label points to the input
Implicit label	Input is inside the label	The label wraps the input

Explicit example:

```
<label for="email">Email Address:</label>
<input type="email" id="email" name="email">
```

Implicit example:

```
<label>
  Email Address:
  <input type="email" name="email">
</label>
```

Beginner recommendation:

Use explicit labels most of the time while learning.
They are easier to check and debug.

17. The `id` Attribute in Forms

The `id` attribute gives an element a unique identifier on the page.

In forms, `id` is often used to connect a label to an input.

Example:

```
<label for="email">Email Address:</label>
<input type="email" id="email" name="email">
```

Here, the input has:

```
id="email"
```

The label uses:

```
for="email"
```

So the label connects to this input.

Important rules for `id`:

- An `id` should be unique on the page
- Do not reuse the same `id` for multiple inputs
- Avoid spaces in `id` values
- Use clear names

Good:

```
<input type="text" id="full-name">
<input type="email" id="email">
```

Bad:

```
<input type="text" id="field">  
<input type="email" id="field">
```

Why bad?

Because the same `id` is used twice.

Simple rule:

`id` identifies one specific element on the page.

18. The `name` Attribute in Forms

The `name` attribute gives the input a data name when the form is submitted.

Example:

```
<input type="email" id="email" name="email">
```

If the user types:

student@example.com

The submitted data may be sent like:

email=student@example.com

Here:

email

comes from:

name="email"

Simple meaning:

name is the key used for the submitted form data.

Another example:

```
<input type="text" id="full-name" name="full-name">
```

If the user types:

Sangeeth Perera

The submitted data may look like:

full-name=Sangeeth Perera

Simple rule:

name identifies the field in submitted data.

19. id vs name

Beginners often confuse `id` and `name`.

They are not the same.

Attribute	Main Purpose	Simple Meaning
<code>id</code>	Connect label to input and identify element on page	Used by the page
<code>name</code>	Identify submitted form data	Used when sending data

Example:

```
<label for="email">Email Address:</label>  
<input type="email" id="email" name="email">
```

Here:

```
id="email"
```

connects with:

```
for="email"
```

And:

```
name="email"
```

identifies the data when the form is submitted.

Simple memory rule:

```
id connects the label.  
name names the submitted data.
```

20. What Is a Placeholder?

A `placeholder` gives a short hint inside an input.

Example:

```
<label for="email">Email Address:</label>  
<input type="email" id="email" name="email" placeholder="example@email.com">
```

The placeholder text is:

```
example@email.com
```

It helps the user understand the expected format.

Good uses of placeholder:

- Showing email format
- Showing example username format
- Showing example phone format
- Giving a short hint

Example:

```
<input type="email" id="email" name="email" placeholder="example@email.com">
```

Simple meaning:

```
Type an email like this example.
```

21. Placeholder Is Not a Label

A placeholder must not replace a real label.

Bad:

```
<input type="email" placeholder="Email Address">
```

Why is this bad?

Because there is no real label.

Problems:

- Placeholder disappears when the user types
- It may be hard to remember what the field was for
- Some users may miss it
- Screen reader support may not be as clear as a proper label
- It is weaker for accessibility

Better:

```
<label for="email">Email Address:</label>
<input type="email" id="email" name="email" placeholder="example@email.com">
```

Here:

- The label says what the field is
- The placeholder gives a small example

Simple rule:

Label = field name
Placeholder = temporary hint

22. Good Placeholder vs Bad Placeholder

Bad placeholder use

```
<input type="text" placeholder="Full Name">
```

Problem:

The placeholder is being used as the only label.

Better version

```
<label for="full-name">Full Name:</label>  
<input type="text" id="full-name" name="full-name">
```

Good placeholder use

```
<label for="email">Email Address:</label>  
<input type="email" id="email" name="email" placeholder="example@email.com">
```

Why this is good:

The label gives the field name.
The placeholder gives a helpful example.

23. Basic Form Anatomy

A simple beginner form can have this structure:

```
<form action="">  
  <label for="full-name">Full Name:</label>  
  <input type="text" id="full-name" name="full-name">
```

```
<label for="email">Email Address:</label>
<input type="email" id="email" name="email" placeholder="example@email.com">
</form>
```

Main parts:

Part	Meaning
form	Groups the form controls
action	Says where the data will be sent
label	Gives a field name
for	Connects label to input id
input	Collects user data
type	Says what kind of data is expected
id	Identifies the input on the page
name	Names the submitted data
placeholder	Gives a temporary hint

24. Full Beginner Registration Form Example

This example uses Sprint 2 boilerplate and Sprint 7 form fundamentals.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Registration Form</title>
</head>
<body>
  <header>
    <h1>Registration Form</h1>
  </header>

  <main>
    <p>Please enter your details below.</p>

    <form action="">
      <label for="full-name">Full Name:</label>
```

```
    <input type="text" id="full-name" name="full-name">

    <label for="email">Email Address:</label>
    <input type="email" id="email" name="email"
placeholder="example@email.com">
  </form>
</main>
</body>
</html>
```

25. Explanation of the Full Example

Boilerplate

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Registration Form</title>
</head>
<body>
  ...
</body>
</html>
```

This creates a complete HTML document.

Header

```
<header>
  <h1>Registration Form</h1>
</header>
```

This gives the page a clear main heading.

Main content

```
<main>
  <p>Please enter your details below.</p>
```

```
...  
</main>
```

The `main` element contains the main page content.

Form

```
<form action="">  
...  
</form>
```

This creates a form area.

The empty `action` means the form is not connected to a real server destination yet.

Full name label and input

```
<label for="full-name">Full Name:</label>  
<input type="text" id="full-name" name="full-name">
```

This creates a text input for the user's full name.

The label is connected because:

```
for="full-name"  
id="full-name"
```

match.

Email label and input

```
<label for="email">Email Address:</label>  
<input type="email" id="email" name="email" placeholder="example@email.com">
```

This creates an email input.

The label is connected because:

```
for="email"  
id="email"
```

match.

The placeholder gives an example email format.

26. What Sprint 7 Does Not Cover Yet

Sprint 7 does not deeply cover:

- Submit buttons
- Reset buttons
- Required fields
- minlength
- maxlength
- Disabled fields
- Readonly fields
- Full validation
- Server-side processing
- JavaScript form behavior

Those topics come later, especially in Sprint 8.

For Sprint 7, your focus is:

Build the basic form structure correctly.
Connect every input to a proper label.
Use placeholder only as a hint.

27. Common Sprint 7 Mistakes and Fixes

Mistake 1: Input has a closing tag

Wrong:

```
<input type="text"></input>
```

Problem:

input is a void element.
It must not have a closing tag.

Correct:

```
<input type="text">
```

Mistake 2: Input has no label

Wrong:

```
<input type="text" id="full-name" name="full-name">
```

Problem:

The user may not know what this field is for.

Correct:

```
<label for="full-name">Full Name:</label>  
<input type="text" id="full-name" name="full-name">
```

Mistake 3: Label text is written as plain text

Weak:

```
Name: <input type="text">
```

Problem:

Name is visible text, but it is not a real label element.

Better:

```
<label for="full-name">Full Name:</label>  
<input type="text" id="full-name" name="full-name">
```

Mistake 4: `for` and `id` do not match

Wrong:

```
<label for="mail">Email:</label>
<input type="email" id="email" name="email">
```

Problem:

`for="mail"` does not match `id="email"`.
The label is not correctly connected.

Correct:

```
<label for="email">Email:</label>
<input type="email" id="email" name="email">
```

Mistake 5: Placeholder used as the only label

Wrong:

```
<input type="email" placeholder="Email">
```

Problem:

The input has no real label.
The placeholder disappears when the user types.

Correct:

```
<label for="email">Email Address:</label>
<input type="email" id="email" name="email" placeholder="example@email.com">
```

Mistake 6: Missing `name` attribute

Weak:

```
<label for="email">Email Address:</label>
<input type="email" id="email">
```

Problem:

The label can connect to the input, but the submitted data does not have a clear field name.

Better:

```
<label for="email">Email Address:</label>
<input type="email" id="email" name="email">
```

Mistake 7: Missing `action` attribute

Weak:

```
<form>
  <label for="email">Email Address:</label>
  <input type="email" id="email" name="email">
</form>
```

Better for Sprint 7 practice:

```
<form action="">
  <label for="email">Email Address:</label>
  <input type="email" id="email" name="email">
</form>
```

Simple reason:

action shows where the form data should go.
For practice, action can be empty.

28. Sprint 7 Debugging Practice

Broken code

```
<form>
  Name: <input type="text"></input>
  <label for="mail">Email</label>
  <input type="email" id="email" placeholder="Email">
</form>
```

Problems

Problem 1:

```
<form>
```

The form has no `action` attribute.

Better:

```
<form action="">
```

Problem 2:

```
Name: <input type="text"></input>
```

Problems:

- `Name:` is not a real label
- `input` has a wrong closing tag
- The input has no `id`
- The input has no `name`

Better:

```
<label for="full-name">Full Name:</label>
<input type="text" id="full-name" name="full-name">
```

Problem 3:

```
<label for="mail">Email</label>
<input type="email" id="email" placeholder="Email">
```

Problems:

- `for="mail"` does not match `id="email"`
- The input has no `name`
- Placeholder is weak because it repeats the label instead of giving an example

Better:

```
<label for="email">Email Address:</label>
<input type="email" id="email" name="email" placeholder="example@email.com">
```

Final fixed version

```
<form action="">
  <label for="full-name">Full Name:</label>
  <input type="text" id="full-name" name="full-name">

  <label for="email">Email Address:</label>
  <input type="email" id="email" name="email" placeholder="example@email.com">
</form>
```

Why this fixed version is correct

This version is correct because:

- The form has an `action` attribute
- Every input has a real label
- `input` elements do not have closing tags
- The label `for` values match the input `id` values
- Both inputs have `name` attributes
- The placeholder is used only as a hint
- The email placeholder shows an example format

29. Sprint 7 Practice Task

Create a file named:

sprint-7-registration-form.html

Build a beginner registration form.

Requirements

Your page should include:

1. Full Sprint 2 boilerplate
2. One `h1` with the text `Registration Form`
3. One paragraph explaining the form
4. One `form` element
5. `action=""` on the form
6. Full Name text input
7. Email Address email input
8. Explicit labels for both inputs
9. Matching `for` and `id` values
10. `name` attributes for both inputs
11. One helpful placeholder for the email input
12. No submit button yet
13. No validation yet
14. No JavaScript
15. Only HTML

30. Starter Practice Code

Use this only after trying to write it yourself first.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Registration Form</title>
</head>
<body>
  <h1>Registration Form</h1>

  <p>Please enter your full name and email address to register.</p>

  <form action="">
    <label for="full-name">Full Name:</label>
    <input type="text" id="full-name" name="full-name">
```

```
<label for="email">Email Address:</label>
<input type="email" id="email" name="email" placeholder="example@email.com">
</form>
</body>
</html>
```

31. Sprint 7 Self-Check Checklist

After writing your Sprint 7 form, check these points:

- Did I write a full HTML document?
- Did I include `<!DOCTYPE html>`?
- Did I include `<html lang="en">`?
- Did I include `head` and `body`?
- Did I include a page title?
- Did I create a form?
- Does the form have `action=""`?
- Does every input have a label?
- Are my labels real `label` elements?
- Does every explicit label have a `for` attribute?
- Does every matching input have the correct `id`?
- Do `for` and `id` values match exactly?
- Does every input have a useful `name` attribute?
- Did I avoid `</input>`?
- Did I use placeholder only as a hint?
- Did I avoid using placeholder as the only label?
- Did I avoid validation for now?
- Did I avoid buttons for now?
- Did I use only HTML?

32. Sprint 7 Revision Questions

Use these questions to test your understanding.

Question 1

What does the `form` element do?

Answer:

The `form` element groups controls that collect user input.

Question 2

What does the `action` attribute specify?

Answer:

The action attribute specifies where the form data will be sent when the form is submitted.

Question 3

Why is `input` called a void element?

Answer:

Because input does not contain content and does not have a closing tag.

Question 4

What does `type="text"` mean?

Answer:

It means the input expects normal text.

Question 5

What does `type="email"` mean?

Answer:

It means the input expects an email address.

Question 6

What does a `label` do?

Answer:

A label gives a name or explanation for an input field.

Question 7

How does explicit label association work?

Answer:

The label uses a `for` attribute, and the input uses a matching `id`.

Example:

```
<label for="email">Email Address:</label>
<input type="email" id="email" name="email">
```

Question 8

How does implicit label association work?

Answer:

The input is placed inside the label element.

Example:

```
<label>
  Email Address:
  <input type="email" name="email">
</label>
```

Question 9

What is the difference between `id` and `name`?

Answer:

`id` identifies the input on the page and helps connect the label.
`name` identifies the field when form data is submitted.

Question 10

Why should placeholder not replace a label?

Answer:

Because placeholder text disappears when the user types, and it is not as clear or accessible as a real label.

Question 11

Why are labels important for screen readers?

Answer:

Screen readers use labels to tell users what each input field is for.

Question 12

What is wrong with this code?

```
<input type="text"></input>
```

Answer:

input is a void element, so it must not have a closing tag.

Correct:

```
<input type="text">
```

33. Sprint 7 Memory Template

Memorize this pattern:

```
<form action="">
  <label for="field-id">Field Label:</label>
  <input type="text" id="field-id" name="field-name">
</form>
```

For email:

```
<form action="">
  <label for="email">Email Address:</label>
  <input type="email" id="email" name="email" placeholder="example@email.com">
</form>
```

Simple memory rule:

```
form groups the fields.
action says where data goes.
label explains the field.
for connects to id.
input collects the data.
type says what kind of data.
id connects the label.
name identifies submitted data.
placeholder gives a hint.
```

34. Final Sprint 7 Summary

Sprint 7 teaches the foundation of HTML forms.

The most important idea is:

```
A form should collect user input clearly and accessibly.
```

The main elements and attributes are:

```
<form action="">
<label for="..."></label>
<input type="..." id="..." name="..." placeholder="...">
```

The strongest beginner pattern is:

```
<label for="full-name">Full Name:</label>
<input type="text" id="full-name" name="full-name">
```

Always remember:

- Use a real `label`

- Connect `label for` to `input id`
- Give inputs a `name`
- Use `placeholder` only as a hint
- Do not write `</input>`
- Do not replace labels with placeholders
- Do not add validation yet in Sprint 7
- Do not add buttons yet if the task says not to
- Keep the form simple and correct

Sprint 7 is complete when you can build a small form with full name and email fields from memory.